

UNIT 2: REGULAR EXPRESSIONS AND LANGUAGES

Course Pattern: TE IT - 2024 Pattern (SPPU)
Status: Comprehensive Master Theory Guide

Academic Term: Semester V (2026)
Course Code: PCC-303-ITT

Syllabus Mapping & Unit Structure

The following table outlines the exam-oriented curriculum mapping and priority weightage for Unit 2.

Syllabus Topic	In-Depth Exam Coverage	Priority / Weightage
Regular Expression (RE) Fundamentals	Formal Definition of RE, Primitive terms ($\emptyset, \varepsilon, a$), Operators (Union $+$, Concatenation $\cdot$, Kleene Star $*$), Identities & Algebraic Laws, Equivalence of REs.	High Priority (4-6 Marks Qs)
Equivalence of RE & Regular Languages	Equivalence theorem, Converting algebraic RE to Regular Languages (RL), Properties of Regular Sets.	High Priority (6-8 Marks Qs)
RE to FA Conversion (Direct Method)	McNaughton-Yamada-Thompson Construction, Direct Conversion of RE to ε -NFA / NFA / DFA.	High Priority (8-10 Marks Qs)
FA to RE Conversion (Arden's Theorem)	Arden's Theorem statement & proof ($R = Q + RP \implies R = QP^*$), System of linear equations method, State Elimination Method.	High Priority (8-10 Marks Qs)
Closure Properties of Regular Languages	Closure under Union, Concatenation, Kleene Star, Intersection, Complement, Difference, Reversal, and Homomorphism.	High Priority (8-10 Marks Qs)
Applications of RE	Password & Input Validation Systems, Pattern Matching, Intrusion Detection Systems (IDS), Lexical Analysis.	Medium Priority (4-6 Marks Qs)

Regular Expressions (RE) and Algebraic Identities

Definition and Operators of Regular Expressions

A Regular Expression (RE) is an algebraic declaration that specifies a regular language over an alphabet Σ .

DEFINITION:

Let Σ be an alphabet. A **Regular Expression** over Σ and the set $L(R)$ of strings it denotes are defined inductively:

Base Cases:

1. $\emptyset$ is a regular expression representing the empty language $L(\emptyset) = \emptyset$.
2. ε is a regular expression representing the language $L(\varepsilon) = \{\varepsilon\}$.
3. For each symbol $a \in \Sigma$, a is a regular expression representing $L(a) = \{a\}$.

Inductive Steps: If r_1 and r_2 are regular expressions denoting $L(r_1)$ and $L(r_2)$ respectively:

1. **Union (Disjunction):** $(r_1 + r_2)$ or $(r_1 \mid r_2)$ denotes $L(r_1) \cup L(r_2)$.
2. **Concatenation:** $(r_1 r_2)$ or $(r_1 c \cdot r_2)$ denotes $L(r_1)L(r_2)$.
3. **Kleene Closure:** (r_1^*) denotes $(L(r_1))^*$.

Precedence of RE Operators

When evaluating regular expressions without explicit parentheses, the standard operator precedence order is:

1. **Kleene Star (*)**: Highest precedence (applies to the immediate preceding character or parenthesized sub-expression).
2. **Concatenation (c ·)**: Second highest precedence.
3. **Union (+ or |)**: Lowest precedence.

Algebraic Identities and Laws of Regular Expressions**NOTE:**

For any regular expressions P, Q, R :

- **Identity for Union:** $P + \emptyset = P$, $P + P = P$ (Idempotence).
- **Identity for Concatenation:** $P\varepsilon = \varepsilon P = P$.
- **Annihilator for Concatenation:** $P\emptyset = \emptyset P = \emptyset$.
- **Commutativity of Union:** $P + Q = Q + P$.
- **Associativity:** $(P + Q) + R = P + (Q + R)$, $(PQ)R = P(QR)$.
- **Distributivity:** $P(Q + R) = PQ + PR$, $(P + Q)R = PR + QR$.
- **Closure Identities:**
 - $\emptyset^* = \varepsilon$, $\varepsilon^* = \varepsilon$.
 - $P^*P^* = P^*$, $(P^*)^* = P^*$.
 - $PP^* = P^*P = P^+$.
 - $(P + Q)^* = (P^*Q^*)^* = (P^* + Q^*)^*$.
 - $P(QP)^* = (PQ)^*P$.

Source: Hopcroft et al., Ch 3, pp. 85–91 | Source: KLP Mishra, Ch 5, pp. 149–153

Equivalence of Regular Expressions and Regular Languages**Definition of Regular Languages (RL)**

A language $L \subseteq \Sigma^*$ is defined as a **Regular Language** if and only if there exists a finite automaton (DFA, NFA, or ε -NFA) that accepts L , or equivalently, if L can be described by a regular expression R .

IMPORTANT:

Equivalence Theorem (Kleene's Theorem): A language L is accepted by a Finite Automaton if and only if L is represented by a Regular Expression.

$$\text{Finite Automata (DFA / NFA)} \iff \text{Regular Language (RL)} \iff \text{Regular Expression (RE)}$$

Source: Hopcroft et al., Ch 3, pp. 92–93 | Source: Sipser, Ch 1, pp. 87–90

Conversion of Regular Expression to Finite Automata

Thompson's Construction (Direct Method for RE to ε -NFA)

Thompson's construction algorithm recursively builds an ε -NFA from a given regular expression R .

DEFINITION:**Base Structural Rules:**

1. For $r = \emptyset$: Initial state q_0 with no transitions to final state q_f .
2. For $r = \varepsilon$: $q_0 \xrightarrow{\varepsilon} q_f$.
3. For $r = a$ ($a \in \Sigma$): $q_0 \xrightarrow{a} q_f$.

Composite Structural Rules:

1. **Union** ($r_1 + r_2$): Introduce new start state q_0 with ε -moves to start states of $N(r_1)$ and $N(r_2)$, and ε -moves from their final states to a new final state q_f .
2. **Concatenation** ($r_1 r_2$): Connect final state of $N(r_1)$ to start state of $N(r_2)$ via ε -transition.
3. **Kleene Star** (r_1^*): Introduce new start state q_0 and final state q_f . Add ε -moves from $q_0 \rightarrow q_f$, $q_0 \rightarrow \text{start}(N(r_1))$, $\text{final}(N(r_1)) \rightarrow q_f$, and loopback $\text{final}(N(r_1)) \rightarrow \text{start}(N(r_1))$.

Source: Hopcroft et al., Ch 3, pp. 102–107 | Source: KLP Mishra, Ch 5, pp. 160–165

Conversion of Finite Automata to Regular Expressions

Arden's Theorem

Arden's Theorem provides a systematic algebraic tool to solve set equations for converting Finite Automata to Regular Expressions.

DEFINITION:

Statement of Arden's Theorem: Let P and Q be two regular expressions over Σ . If P does not contain the empty string ε ($\varepsilon \notin L(P)$), then the linear equation in R :

$$R = Q + RP$$

has a **unique solution** given by:

$$R = QP^*$$

NOTE:**Proof of Arden's Theorem:**

1. **Existence:** Substitute $R = QP^*$ into RHS:

$$\text{RHS} = Q + (QP^*)P = Q(\varepsilon + P^*P) = QP^* = \text{LHS}$$

2. **Uniqueness:** Repeatedly substitute $R = Q + RP$ into itself:

$$R = Q + (Q + RP)P = Q + QP + RP^2 = Q(\varepsilon + P + P^2 + \dots + P^k) + RP^{k+1}$$

As $k \rightarrow \infty$, $Q(\varepsilon + P + P^2 + \dots) = QP^*$. Since $\varepsilon! \in L(P)$, RP^{k+1} contains strings of length at least $k + 1$, contributing nothing to finite strings denoted by R . Thus $R = QP^*$.

Conversion of FA to RE using Arden's Theorem (System of Linear Equations)

Given a DFA/NFA with states $q_1, q_2, \dots, q_n$ where q_1 is the initial state:

1. Write a state transition equation for each state q_i :

$$q_i = \sum_j q_j w_{\{ji\}} + (\text{add } \varepsilon \text{ if } q_i \text{ is initial state } q_1)$$

where $w_{\{ji\}}$ is the symbol labeling the transition from q_j to q_i .

2. Solve the system of algebraic equations for state variables using Arden's Theorem ($R = Q + RP \implies R = QP^*$).
3. The overall regular expression for the FA is the sum of expressions for all final states $q_f \in F$:

$$R_{\text{total}} = \sum_{q_f \in F} q_f$$

State Elimination Method

An alternative structural method to convert FA to RE:

1. Construct a Generalized Transition Graph (GTG) with single initial state q_0 and single final state q_f .
2. Eliminate intermediate states one by one. When eliminating state q_k , update transition labels between remaining states p and q via:

$$R'_{\{pq\}} = R_{\{pq\}} + R_{\{pk\}}(R_{\{kk\}})^* R_{\{kq\}}$$

3. Repeat until only q_0 and q_f remain. The final edge label is the regular expression.

Source: KLP Mishra, Ch 5, pp. 152–158 | Source: Hopcroft et al., Ch 3, pp. 93–98

Closure Properties of Regular Languages

Regular languages are closed under various operations, meaning applying these operations on regular languages yields another regular language.

IMPORTANT:**Summary of Closure Properties:**

1. **Union:** If L_1, L_2 are regular, $L_1 \cup L_2$ is regular (by parallel FA construction or $R_1 + R_2$).
2. **Concatenation:** If L_1, L_2 are regular, $L_1 L_2$ is regular (by sequential FA construction or $R_1 R_2$).
3. **Kleene Closure:** If L is regular, L^* is regular.
4. **Complement:** If L is regular over Σ , $L^c = \Sigma^* - L$ is regular (swap final & non-final states in DFA).
5. **Intersection:** If L_1, L_2 are regular, $L_1 \cap L_2$ is regular (by Product Automaton construction or De Morgan's Law: $L_1 \cap L_2 = (L_1^c \cup L_2^c)^c$).
6. **Difference:** $L_1 - L_2 = L_1 \cap L_2^c$ is regular.
7. **Reversal:** $L^R = \{w^R : w \in L\}$ is regular (reverse all transition arrows in FA and swap initial & final states).
8. **Homomorphism:** If $h : \Sigma \rightarrow \Gamma^*$ is a homomorphism, $h(L)$ is regular.

Source: Hopcroft et al., Ch 4, pp. 133–148 | Source: KLP Mishra, Ch 5, pp. 170–178

Real-World Applications of Regular Expressions

1. **Password & Input Validation Systems:**
 - Form field validation (e.g. email verification, strong password criteria) via regex engines.
2. **Intrusion Detection Systems (IDS) & Log Analysis:**
 - Deep packet inspection engines and security audit tools (e.g. Snort, grep) use regex pattern matching to detect signature vectors and unauthorized access logs.
3. **Lexical Analysis in Compilers:**
 - Standard lexer tools (`flex`, `lex`) match token streams (identifiers, literals, operators) using compiled minimal DFAs derived from regular expressions.

Source: Hopcroft et al., Ch 3, pp. 109–114 | Source: SPPU Syllabus PCC-303-ITT, p. 20